

Traduction des langages

Réalisation d'un compilateur pour le langage RAT

Préambule

- Les TP sont à réaliser en binôme. Le projet consistera à enrichir le langage RAT, il s'appuiera donc sur le travail réalisé en séance de TP. Les binômes de projet seront donc les binômes de TP.
- Les TP s'enchaînent sans que des corrections des étapes intermédiaires ne soient données. Il faut donc, au besoin, finir le travail entre deux séances de TP.
- Un style de programmation fonctionnelle PURE est demandé. Les seuls effets de bord autorisés sont ceux sur les informations associées à un identificateur.

ATTENTION : l'ordre d'évaluation d'Ocaml n'est pas intuitif (dernier paramètre d'un n-uplet en premier, dernier paramètre d'une fonction en premier, opérande droite puis opérande gauche, ...). Ceci n'a aucun impact en programmation fonctionnelle pure, mais en a dès qu'il y a des effets de bord. S'il y a des effets de bord dans `xxx` et `yyy`, privilégiez une écriture de la forme :

```
let e1 = xxx in
```

```
let e2 = yyy in
```

```
(e1,e2)
```

où c'est vous qui choisissez l'ordre l'évaluation (ordre des `let ... in`), plutôt que

```
(xxx,yyy)
```

qui laisse à OCaml le choix de l'ordre d'évaluation (`yyy` sera évalué en premier).

- Séance 1 : Résolution des identifiants
- Séance 2 : Typage
- Séance 3 : Placement mémoire
- Séances 4 et 5 : Génération de code

1 Grammaire formelle du langage Rat

- | | |
|---|---------------------------------|
| 1. $PROG' \rightarrow PROG$ | 20. $TYPE \rightarrow int$ |
| 2. $PROG \rightarrow FUN\ PROG$ | 21. $TYPE \rightarrow rat$ |
| 3. $FUN \rightarrow TYPE\ id\ (DP)\ BLOC$ | 22. $E \rightarrow id\ (CP)$ |
| 4. $PROG \rightarrow id\ BLOC$ | 23. $CP \rightarrow$ |
| 5. $BLOC \rightarrow \{ IS \}$ | 24. $CP \rightarrow E\ CP2$ |
| 6. $IS \rightarrow I\ IS$ | 25. $CP2 \rightarrow ,\ E\ CP2$ |
| 7. $IS \rightarrow$ | 26. $CP2 \rightarrow$ |
| 8. $I \rightarrow TYPE\ id = E ;$ | 27. $E \rightarrow [E / E]$ |
| 9. $I \rightarrow id = E ;$ | 28. $E \rightarrow num\ E$ |
| 10. $I \rightarrow const\ id = entier ;$ | 29. $E \rightarrow denom\ E$ |
| 11. $I \rightarrow print\ E ;$ | 30. $E \rightarrow id$ |
| 12. $I \rightarrow if\ E\ BLOC\ else\ BLOC$ | 31. $E \rightarrow true$ |
| 13. $I \rightarrow while\ E\ BLOC$ | 32. $E \rightarrow false$ |
| 14. $I \rightarrow return\ E ;$ | 33. $E \rightarrow entier$ |
| 15. $DP \rightarrow$ | 34. $E \rightarrow (E + E)$ |
| 16. $DP \rightarrow TYPE\ id\ DP2$ | 35. $E \rightarrow (E * E)$ |
| 17. $DP2 \rightarrow ,\ TYPE\ id\ DP2$ | 36. $E \rightarrow (E = E)$ |
| 18. $DP2 \rightarrow$ | 37. $E \rightarrow (E < E)$ |
| 19. $TYPE \rightarrow bool$ | 38. $E \rightarrow (E)$ |

2 Analyses lexicale et syntaxique et génération de l'arbre abstrait

Les analyses lexicales et syntaxiques, ainsi que la génération de l'arbre abstrait, sont fournies. Elles sont réalisées avec à l'aide de Menhir (analyseur LR).

1. Le fichier `ast.ml` contient :
 - une interface qui donne la structure générale des arbres qui seront parcourus et créés par les différentes passes du compilateur ;
 - une interface d'affichage de ces arbres ;
 - un module `AstSyntax` contenant la définition du type des arbres abstraits après l'analyse lexicale et syntaxique.
 - un module `AstTds` contenant la définition du type des arbres abstraits après la passe de gestion des identifiants vu en TD. Le module d'affichage n'est pas fourni et il n'est pas indispensable de le réaliser.

Il ouvre le module `Type` pour la définition des types fournis dans le langage. Il sera à compléter pour ajouter les structures des arbres pour les différentes passes. L'implantation d'un afficheur pour chaque type d'arbre est optionnelle.
2. Le fichier `lexer.mll` réalise l'analyse lexicale.
3. Le fichier `parser.mly` réalise l'analyse syntaxique et la construction de l'arbre abstrait.

3 Analyse sémantique : réalisation du compilateur par passes

Lorsque l'analyse syntaxique du fichier est réalisée sans erreur, on obtient l'arbre abstrait. Les traitements sémantiques souhaités seront réalisés par passes successives qui effectueront des vérifications sur la structure du programme (gestion des identifiants et typage), et transformeront l'arbre.

La structure générale du compilateur par passes vous est fournie.

- Le fichier `passe.ml` contient :
 - une interface `Passe` qui spécifie qu'une passe est composée de deux types et d'une fonction d'analyse qui prend une expression du premier type et renvoie une expression du second type ;
 - différents modules, conformes à l'interface `Passe`, qui correspondent à des passes "qui ne font rien" (ils seront nécessaires pour tester votre compilateur au fur et à mesure de l'écriture des passes).
- Le fichier `compilateur.ml` contient :
 - un foncteur `Compilateur`, paramétré par quatre `Passe`, spécifiant qu'un compilateur est l'application successive des quatre passes (gestion des identifiants, typage, placement mémoire et génération de code) ;
 - cinq modules instanciant le foncteur `Compilateur` et permettant de réaliser des tests après l'implantation successive des passes.

4 Tests

Plusieurs jeux de tests vous sont fournis dans le répertoire `tests`.

- `dune runtest tests/XXX/sans_fonction` lancera les tests de la passe XXX avec des programmes Rat qui n'ont pas de fonction.
- `dune runtest tests/XXX/avec_fonction` lancera les tests de la passe XXX avec des programmes Rat qui ont des fonctions.
- `dune runtest tests/XXX` lancera les deux jeux de tests précédents.
- Valeurs pour XXX
 - passe de gestion d'identifiants → `gestion_id`
 - passe de typage → `type`
 - passe de placement mémoire → `placement`

- passe de génération de code $\rightarrow$ tam
- `dune runtest` lancera les tests de toutes les passes sur des fichiers Rat sans et avec fonctions.
- `dune runtest tests/*/sans_fonction` lancera les tests de toutes les passes sur des fichiers Rat sans fonction.

5 Implantation des passes

5.1 Passe de résolution des identifiants (1 séance)

L'objectif de la séance est de réaliser la passe de résolution des identifiants.

Fichiers fournis :

- un module `Tds` contenant la structure de données représentant la table des symboles ;
- un fichier `passeTdsRat.ml` contenant un module conforme à l'interface `Passe` et contenant le code de résolution des identifiants pour les blocs et les instructions.

Il n'est normalement pas nécessaire de modifier le module `Tds`, mais vous être libre de le faire si vous en ressentez le besoin.

Il est important d'utiliser les exceptions fournies dans `exceptions.ml` pour que les tests fournis aient du sens.

Vous devez :

1. Compléter la passe de résolution des identifiants (module `PasseTdsRat` conforme à `Passe` dans le fichier `passeTdsRat.ml`) en procédant par étapes. Par exemple :
 - (a) Traiter toutes les expressions qui ne sont pas en rapport avec les fonctions (donc ni les déclarations de fonction, ni les appels) ;
 - (b) Tester avec le jeu de test fourni `dune runtest tests/gestion_id/sans_fonction` ;
 - (c) Traiter les fonctions ;
 - (d) Tester avec le jeu de test fourni `dune runtest tests/gestion_id/avec_fonction`.
2. Valider votre implantation à l'aide des tests fournis `dune runtest tests/gestion_id`.

5.2 Passe de typage (1 séance)

L'objectif de la séance est de réaliser la passe de typage.

Fichiers fournis :

- un module `Type` contenant les fonctions nécessaires à la manipulation des types.

Il n'est normalement pas nécessaire de modifier le module `Type`, mais vous être libre de le faire si vous en ressentez le besoin.

Il est important d'utiliser les exceptions fournies dans `exceptions.ml` pour que les tests fournis aient du sens.

Vous devez :

1. Écrire la passe de typage (module `PasseTypeRat` conforme à `Passe`) dans un fichier `passeTypeRat.ml` en procédant par étapes (par exemple sans traiter les fonctions dans un premier temps).
2. Valider votre implantation à l'aide des tests fournis `dune runtest tests/type` (+ `dune runtest tests/gestion_id` comme tests de non-régression).

5.3 Passe de placement mémoire (1 séance)

L'objectif de la séance est de réaliser la passe de placement mémoire. Les informations de placement sont ajoutées à la TDS.

Vous devez :

1. Écrire la passe de placement mémoire (module `PassePlacementRat` conforme à `Passe`) dans un fichier `passePlacementRat.ml` en procédant par étapes (par exemple sans traiter les fonctions dans un premier temps).
2. Valider votre implantation à l'aide des tests fournis `dune runtest tests/placement` (+ tests précédents comme tests de non-régression).

5.4 Passe de génération du code (2 séances)

L'objectif des deux dernières séances est de réaliser la passe de génération de code. Cette dernière passe produit directement une chaîne de caractères, il est inutile de définir une nouvelle structure d'arbre.

Fichiers fournis :

- un module `Code` contenant des fonctions auxiliaires nécessaires à la génération de code ;
- un module `Tam` contenant des fonctions de service pour générer des instructions TAM.

Il n'est normalement pas nécessaire de modifier les modules `Code` et `Tam`, mais vous être libre de le faire si vous en ressentez le besoin.

Vous devez :

1. Écrire la passe de génération de code (module `PasseCodeRatToTam` conforme à `Passe`) dans un fichier `passeCodeRatToTam.ml` en procédant par étapes (par exemple sans traiter les fonctions dans un premier temps).
2. Tester au fur et à mesure en exécutant le code généré à l'aide d'`itam` (cf Moodle) .
3. Valider votre implantation à l'aide des tests fournis `dune runtest tests/tam` (+ tests précédents comme tests de non-régression).